

BUỔI 9: Hàm

I. Lý thuyết

1. Định nghĩa

Hàm (function) là một đoạn chương trình thực hiện một tác vụ được định nghĩa cụ thể.

Ưu điểm của việc sử dụng hàm trong chương trình:

- Cấu trúc chương trình rõ ràng
- Dễ bảo trì vì khi sửa lỗi thường sẽ sửa từng hàm trong chương trình
- Tiết kiệm chi phí viết code (chỉ viết hàm một lần nhưng có thể sử dụng nhiều lần trong chương trình)

2. Cú pháp

```
data_type function_name (formal argument)
{
    //Body of the function
}
```

Trong đó:

- `data_type`: xác định kiểu giá trị trả về của hàm
- + Nếu hàm không có giá trị trả về: `void`
- + `data_type` là không bắt buộc nếu giá trị trả về của hàm là một số nguyên hoặc không có giá trị trả về.
- `function_name`: tên hàm (đặt theo quy tắc đặt tên biến)
- `formal arguments`: `type1 argument1, type2 argument2, ...`
Hàm có thể không có tham số
`Argument1, argument2, ...` được gọi là các tham số hình thức, các tham số được phân tách với nhau bởi dấu phẩy.
- `Body of the function`: thân của hàm, nếu hàm có giá trị trả về thì trong thân hàm cần có câu lệnh `return expression;`
 - Kiểu giá trị của `expression` chính là `data_type` của hàm
 - Câu lệnh `return` có thể nằm ở bất kỳ vị trí nào của hàm
 - Khi gặp lệnh `return` thì chương trình sẽ thoát khỏi hàm, giá trị mà hàm trả về chính là giá trị của biểu thức sau lệnh `return`.
- Vị trí khai báo hàm: trước hoặc sau `main`

Ví dụ: Viết hàm kiểm tra một số có là nguyên tố hay không

```
int KTNT(int n)
{
    int i;
    if (n<2) return 0;
    else
        if (n==2) return 1;
        else
            if (n%2==0) return 0;
            else
            {
                for (i=2;i<=sqrt(n);i++)
                    if (n%i==0) return 0;
                return 1;
            }
}
```

3. Lời gọi hàm

function _name (actual arguments);

actual arguments: được gọi là tham số thực sự của hàm.

Ví dụ: với hàm kiểm tra nguyên tố ở trên, ta có thể gọi hàm kiểm tra trong main như sau:

```
int main()
{
    int n;
    printf("\nNhập n:");
    scanf("%d", &n);
    if (KTNT(n)==1)
        printf("%d là số nguyên tố", n);
    else
        printf("%d không là nguyên tố", n);

    return 0;
}
```

4. Biến toàn cục và biến cục bộ

- Biến cục bộ:
 - Là biến được khai báo bên trong một hàm.
 - Phạm vi sử dụng: trong thân hàm khai báo biến
 - Thời gian tồn tại: trong thời gian hàm được gọi tới
 - Cấp phát bộ nhớ: được cấp phát khi chương trình bắt đầu thực thi khối lệnh mà các biến cục bộ được khai báo bên trong nó
 - Các biến cục bộ tự động giải phóng bộ nhớ khi hàm kết thúc.

Chú ý:

- Danh sách tham số hình thức của hàm cũng là biến cục bộ
- Biến trong hàm main cũng là biến cục bộ
- Biến toàn cục:
 - Là biến được khai báo bên ngoài các hàm (kể cả hàm main)
 - Thời gian tồn tại: trong suốt thời gian chạy chương trình
 - Cấp phát bộ nhớ: được cấp phát khi chương trình bắt đầu thực hiện
 - Các giá trị được lưu trong biến toàn cục sẽ tồn tại trong suốt thời gian thực thi chương trình.

Chú ý: Khi biến cục bộ và biến toàn cục trùng tên, biến cục bộ sẽ che khuất biến toàn cục.

5. Truyền tham số cho hàm: Truyền tham trị và truyền tham biến

+ Truyền tham trị:

- Là truyền vào hàm giá trị của tham số thực sự, khi đó hàm sẽ tự động tạo ra một địa chỉ ô nhớ mới để lưu trữ giá trị này.
- Mọi sự thay đổi giá trị của biến bên trong thân hàm không ảnh hưởng đến giá trị của biến bên ngoài hàm.
- Việc truyền tham trị hay tham chiếu được quy định khi ta định nghĩa danh sách các tham số hình thức của hàm.

Ví dụ:

```

void HoanVi(int a, int b)
{
    int tg=a;
    a=b;
    b=tg;
}
int main()
{
    int x=10, y=20;
    HoanVi(x,y); //Gọi hàm hoán vị giá trị của x và y
    printf("x=%d, y=%d",x,y); //In giá trị của x và y
    return 0;
}

```

+ Truyền tham biến:

- Là truyền vào hàm địa chỉ của tham số thực sự. Do đó, khi thay đổi giá trị của biến bên trong thân hàm thì giá trị của biến bên ngoài hàm cũng thay đổi theo.
- Truyền tham chiếu trong C sử dụng con trỏ để khai báo danh sách tham số hình thức.
- Truyền tham chiếu được sử dụng khi muốn thay đổi giá trị của biến truyền vào và muốn lưu lại sự thay đổi này ra ngoài hàm.

Ví dụ:

```

void HoanVi(int *a, int *b)
{
    int tg=*a;
    *a=*b;
    *b=tg;
}
int main()
{
    int x=10, y=20;
    HoanVi(&x,&y); //Gọi hàm hoán vị giá trị của x và y
    printf("x=%d, y=%d",x,y); //In giá trị của x và y
    return 0;
}

```

II. Ví dụ

Ví dụ 1:Viết chương trình nhập vào mảng 1 chiều, in ra các số nguyên tố của mảng, tính tổng các số nguyên tố đó.

Hướng dẫn:

```

void Nhap(int A[], int n)
{
    int i;

    for (i=0;i<n;i++)
    {
        printf("A[%d]=",i);
        scanf("%d",&A[i]);
    }
}
//hàm in ma trận
void In(int A[], int n)
{

```

```

    int i;
    printf("Mang da nhap: \n");
    for (i=0;i<n;i++)
        printf("%5d",A[i]);
}
//Hàm kiểm tra số nguyên tố
int check (int k)
{
    int i;
    if (k <2) return 0;
    else
        if (k==2) return 1;
        else
            if (k%2==0) return 0;
            else
            {
                for (i=2;i<=sqrt(k);i++)
                    if (k%i==0) return 0;
                return 1;
            }
}

int main()
{
    int A[max],n,i,T=0;
    printf("Nhap so phan tu:");
    scanf("%d",&n);
    Nhap(A,n);
    In(A,n);
    printf("\nCac so nguyen to cua mang:\n");
    for (i=0;i<n; i++)
        if (check(A[i])==1)
        {
            printf("%5d",A[i]);
            T=T+A[i];
        }
    printf("\nTong cac so nguyen to: %d",T);

    return 0;
}

```

Ví dụ 2: Viết chương trình tìm tổng của hai ma trận bằng cách sử dụng hàm in ma trận, hàm nhập ma trận

Hướng dẫn:

//Nhập ma trận

```

void Nhap(float A[][max], int n, int m)
{
    int i,j;

```

```

        for (i=0;i<n;i++)
            for (j=0;j<m;j++)
            {
                printf("[%d,%d]=",i,j);
                scanf("%f",&A[i][j]);
            }
    }
//In ma tran
void In(float A[][max], int n, int m)
{
    int i,j;
    for (i=0;i<n;i++)
    {
        for (j=0;j<m;j++)
            printf("%7.2f",A[i][j]);
        printf("\n");
    }
}

int main()
{
    int n1,m1,n2,m2,i,j;
    float A[max][max],B[max][max],C[max][max];
    //Nhap ma tran 1
    printf("Nhap thong tin ma tran 1:\n");
    printf("Nhap so hang:");scanf("%d",&n1);
    printf("Nhap so cot:");scanf("%d",&m1);
    Nhap(A,n1,m1);
    //Nhap ma tran 2
    printf("Nhap thong tin ma tran 1:\n");
    printf("Nhap so hang:");scanf("%d",&n2);
    printf("Nhap so cot:");scanf("%d",&m2);
    Nhap(B,n2,m2);
    //Tinh tong hai ma tran
    for (i=0;i<n1;i++)
        for (j=0;j<m1;j++)
            C[i][j]=A[i][j]+B[i][j];
    //In ma tran tong
    printf("Ma tran A: \n");
    In(A,n1,m1);
    printf("Ma tran B: \n");
    In(B,n2,m2);
    printf("Ma tran tong: \n");
    In(C,n1,m1);
    return 0;
}

```

III. Bài tập (yêu cầu sinh viên tự code)

Câu 30: Viết chương trình con đổi một số nguyên dương ở hệ thập phân sang dạng biểu diễn ở cơ số a . Sau đó gọi hàm để đổi một số nhập vào từ bàn phím sang dạng biểu diễn ở các cơ số: 2, 8, 16.

Câu 31: Viết chương trình nhập vào dãy số thực, sắp xếp dãy theo chiều tăng /giảm dần. Sử dụng hàm.

Câu 32: Viết chương trình tính tổng các số hoàn hảo trong ma trận hai chiều có n hàng , m cột.